

ESP32 Bluetooth RC Control System

Miguel Chip

January 29, 2026

Abstract

This document describes the design and implementation of a Bluetooth-based RC controller using the ESP32 microcontroller.

Contents

1	Introduction	2
2	System Overview	3
3	Communication Protocol	4
3.1	Packet Structure	4
4	Main Program	5
4.1	Initialization	5
4.2	Main Loop	5
5	Conclusion	7
A	Full Source Code	8

Chapter 1

Introduction

This project controls RC vehicles using Bluetooth communication.

Chapter 2

System Overview

The ESP32 receives control packets and drives motors using PWM.

Chapter 3

Communication Protocol

3.1 Packet Structure

Listing 3.1: RC Packet Structure

```
typedef struct __attribute__((packed)) {  
    byte startByte1;  
    byte startByte2;  
    uint16_t leftStickX;  
    uint16_t leftStickY;  
    uint16_t rightStickX;  
    uint16_t rightStickY;  
    byte checksum;  
} RcPacket;
```

Each field represents a control channel.

Chapter 4

Main Program

4.1 Initialization

Listing 4.1: ESP32 setup function

```
void setup() {  
    Serial.begin(115200);  
    initBluetooth();  
}
```

4.2 Main Loop

Listing 4.2: Main loop

```
void loop() {  
    if (receivePacket()) {  
        applyControls();  
    }  
}
```

```
1 void loop() {  
2     if (receivePacket()) {  
3         applyControls();  
4     }  
5 }
```

Listing 4.3: Main loop

```
1 int myNum = 5;           // Integer (whole number without decimals)  
2 double myFloatNum = 5.99; // Floating point number (with decimals)  
3 char myLetter = 'D';    // Character  
4 string myText = "Hello"; // String (text)  
5 bool myBoolean = true;
```

Listing 4.4: ESP32 Main Program

Chapter 5

Conclusion

This system enables low-latency Bluetooth RC control.

Appendix A

Full Source Code

```
int myNum = 5;           // Integer (whole number without decimals)
double myFloatNum = 5.99; // Floating point number (with decimals)
char myLetter = 'D';     // Character
string myText = "Hello"; // String (text)
bool myBoolean = true;
```